

ThreatLens

Security Assessment Platform

World Monitor Security Assessment Report

CONFIDENTIAL - FOR AUTHORIZED RECIPIENTS ONLY

Project:	World Monitor
Target:	https://github.com/koala73/worldmonitor
Report Date:	September 17, 2026
Methodology:	Static Analysis . Dynamic Testing . Manual Validation
Prepared By:	ThreatLens Automated Scanner v1.0
Classification:	Confidential

1. EXECUTIVE SUMMARY

Overall Security Posture: HIGH RISK

5

TOTAL FINDINGS

0

CRITICAL

1

HIGH

3

MEDIUM

1

LOW

This automated security assessment identified 5 potential security issue(s) across 3 security category/categories. Of these, 5 remain open and 0 have been resolved. Critical and High severity findings should be addressed immediately.

Assessment Date: September 17, 2026

Target:

2. SEVERITY BREAKDOWN

Severity	Count	% of Total	Status
CRITICAL	0	0%	-
HIGH	1	20%	CONFIRMED
MEDIUM	3	60%	CONFIRMED
LOW	1	20%	CONFIRMED
INFO	0	0%	-

3. ASSESSMENT SCOPE & TARGET

Project Name: World Monitor

Description:

4. METHODOLOGY

ThreatLens performs a multi-phase security assessment combining static source-code analysis, dynamic application testing against authorized local targets, and automated validation. All testing is conducted in a non-destructive manner against explicitly authorized targets only. No testing is performed against third-party infrastructure, production systems outside the defined scope, or without explicit authorization.

Static Analysis:

Source code, configuration files, dependency manifests, and secret patterns are scanned without executing the application.

Dynamic Testing:

HTTP security headers, API endpoints, XSS reflection, and injection vectors are tested against a local authorized instance.

Validation:

Each finding is evaluated for confidence (CONFIRMED / LIKELY / POSSIBLE / FALSE_POSITIVE) using scanner-specific validation logic.

Evidence Collection:

Request/response pairs, code snippets, and configuration excerpts are captured as structured evidence.

Remediation Tracking:

Findings progress through a defined lifecycle: DETECTED -> CONFIRMED -> REMEDIATION -> RETESTING -> RESOLVED.

5. FINDINGS

F01 Dependency: image-size <=2.0.2 - DoS via Infinite Loop in Image

HIGH

CONF

Confirmed

Location: package-lock.json:1

Description

Two HIGH-severity CVEs in transitive dependency image-size <=2.0.2 via chain: @deck.gl/core (direct prod dep) -> @loaders.gl/textures -> texture-compressor -> image-size. GHSA-w3rx-r6r6-pgpr: ICNS parser enters infinite loop on crafted files. GHSA-5p2g-fcmc-qvqq: JXL and HEIF parsers enter infinite loops. If user-supplied images reach the loaders.gl texture pipeline at runtime, an attacker can hang the Node.js process. In current deployment texture-compressor is used primarily at build time, reducing runtime exposure.

Remediation

1. Update image-size to >=2.0.3 or add npm package override. 2. Update @loaders.gl/textures and texture-compressor to patched versions. 3. Validate MIME type before passing images to deck.gl loaders.

Evidence

[DEPENDENCY_RECORD] npm audit - image-size HIGH severity (GHSA-w3rx-r6r6-pgpr, GHSA-5p2g-fcmc-qvqq)

```
npm audit output:
image-size <=2.0.2 Severity: high
- ICNS parser DoS (GHSA-w3rx-r6r6-pgpr)
- JXL/HEIF parser DoS (GHSA-5p2g-fcmc-qvqq)
Chain: @deck.gl/core -> @loaders.gl/textures -> texture-compressor -> image-size
Fixed in: image-size >=2.0.3
```

F02 Dependency: stream-json - O(depth^2) DoS via Deeply Nested JSON

MEDIUM

CONF

Confirmed

Location: package-lock.json:1

Description

GHSA-528h-pc64-c93x: stream-json pick/ignore/filter/replace filters have O(depth^2) complexity on deeply nested JSON. A small crafted JSON with hundreds of nesting levels blocks the Node.js event loop for seconds to minutes (DoS). Risk if user-controlled JSON reaches stream-json filters in Vercel Edge functions or Convex backend handlers.

Remediation

1. Upgrade stream-json to patched version. 2. Enforce JSON depth limits (max 20) before stream-json pipelines. 3. Audit call sites for user-controlled JSON input.

Evidence

[DEPENDENCY_RECORD] npm audit - stream-json O(depth^2) DoS (GHSA-528h-pc64-c93x)

```
npm audit:
stream-json Severity: moderate
Event loop blocked seconds-to-minutes on crafted deeply-nested JSON
https://github.com/advisories/GHSA-528h-pc64-c93x
```

F03 Dependency: uuid v3/v5/v6 - Missing Buffer Bounds Check on C

MEDIUM

CONF

Confirmed

Location: package-lock.json:1

Description

GHSA-w5hq-g745-h8pq: uuid v3/v5/v6 generation functions do not validate caller-supplied buf argument length, potentially causing out-of-bounds writes. Standard usage without custom buf is unaffected. Risk is limited to code paths that accept externally-sourced buffers for UUID generation.

Remediation

1. Upgrade uuid to patched version. 2. Audit uuid.v3/v5/v6 call sites for external buf arguments accepted from user input.

Evidence

[DEPENDENCY_RECORD] npm audit - uuid missing bounds check (GHSA-w5hq-g745-h8pq)

```
npm audit:
  uuid Severity: moderate
  Missing buffer bounds check in uuid.v3/v5/v6 when buf is provided
  https://github.com/advisories/GHSA-w5hq-g745-h8pq
```

F04 Potential: MCP Proxy DNS Rebinding Race (TOCTOU) on Vercel

MEDIUM

CONF

Confirme

Location: api/mcp-proxy.ts:329

Description

MCP proxy validates targets via DNS pre-resolution (assertServerUrlSafe) and revalidateBeforeFetch before fetch(). Vercel Edge Runtime lacks socket-level connection pinning, leaving a TOCTOU gap. Attacker with DNS control: register domain to public IP, pass check, flip DNS to private IP (e.g. 169.254.169.254), fetch() connects to private IP. Code explicitly acknowledges this limitation at line 360. The double-check and disabled cross-origin redirects narrow but do not eliminate the race window.

Remediation

1. Route MCP fetches through Railway relay with socket-level IP pinning. 2. Add hostname allowlist as defense-in-depth. 3. Log/alert on unexpected outbound hostnames. 4. Apply Cloudflare DoH pinning to MCP proxy DNS resolution.

Evidence

[CODE_SNIPPET] Code comment at api/mcp-proxy.ts:360 acknowledging TOCTOU limitation

```
api/mcp-proxy.ts lines 360-365 (verbatim):
// Vercel Edge fetch does not expose a Node-style lookup/socket hook,
// so there is no way to guarantee that the IP we validated is the IP
// fetch() ultimately connects to; a DNS answer can change between our
// resolve and fetch's own resolve.
assertServerUrlSafe() at line 329
revalidateBeforeFetch() at line 368 (best-effort second check)
```

F05 Informational: GitHub Actions CI Uses Default PostgreSQL Credentials

LOW

CONF

Confirme

Location: .github/workflows/test.yml:553

Description

CI workflow hardcodes postgres:postgres (official PostgreSQL Docker image factory defaults) for a local integration test database (UMAMI_TEST_DATABASE_URL). Not a production credential - database is ephemeral, bound to 127.0.0.1, accessible only within GitHub Actions runner. Best-practice deviation: default credentials in version-controlled CI configuration.

Remediation

1. Generate random credentials via GitHub Actions secrets. 2. Use `${{ secrets.CI_DB_PASSWORD }}` for ephemeral test databases.
3. Add comment documenting CI-only scope if static credentials are retained.

Evidence

[CONFIG_VALUE] Hardcoded postgres:postgres in .github/workflows/test.yml:553

```
.github/workflows/test.yml lines 551-558:
```

```
UMAMI_TEST_DATABASE_URL: postgresql://postgres:postgres@127.0.0.1:5432/umami_integration  
(used in two consecutive test steps)
```

Note: postgres:postgres are the official PostgreSQL Docker image default credentials.
CI-only ephemeral test database, not a production credential.

6. SECURITY POSTURE & CONCLUSION

Rating: HIGH RISK

Assessment conducted by ThreatLens Automated Security Platform

1 HIGH severity finding(s) pose significant risk and should be remediated promptly.

NOTE: This report was generated by ThreatLens automated scanning. Results should be reviewed by a qualified security professional before making security decisions. Automated tools cannot detect every vulnerability type.

ThreatLens . Authorized Security Assessment Platform . Confidential